NAME:

**Project 4: Sentiment Analysis (5 pts)**

Sentiment analysis seeks to determine the general attitude of a writer, given some text they have written. For example, given the movie review "The film was a breath of fresh air" a sentiment analysis program should realize that this is a positive statement about the movie, while the review "It made me want to poke out my eye balls" expresses a negative opinion. In this project you'll implement a very simple sentiment analysis system for movie reviews.

**Learning Objectives:**

1. Demonstrate reading input from files.
2. Practice using dictionaries.
3. Explore sentiment analysis.

**Reminder:** Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Please see the syllabus or Nexus for what to do when you need help with your work.

**Movie review dataset**

You can download three files with movie reviews from the course website. The movie reviews come from Rotten Tomatoes and their sentiment has been manually rated on a scale from 0 to 4.

- 0 - negative
- 1 - somewhat negative
- 2 - neutral
- 3 - somewhat positive
- 4 - positive

Each file is formatted as follows:

```
0 Devoid of any of the qualities that made the first film so special .
4 The Bai brothers have taken an small slice of history and opened it up for all ...
3 Ramsay and Morton fill this character study with poetic force and buoyant ...
1 You emerge dazed , confused as to whether you 've seen pornography or documentary .
```

There is one review per line. Each line starts with the review's sentiment score, followed by the words of the review. The words have already been pre-processed so that all words and punctuation symbols are separated by a blank character.

You are given the following three files with movie reviews:

- `movie_reviews_training.txt` contains 6129 movie reviews. Use this to build up a dictionary of sentiment scores for each word. (See further explanation below.)
- `movie_reviews_mini.txt` contains only the first 15 movie reviews from `movie_reviews_training.txt`. Use this file to test our your function for building a word sentiment dictionary on a more manageable file.

- `movie_reviews_dev.txt` contains 800 reviews. Use this to evaluate how well your sentiment analysis system works and to fine-tune it.

**General Approach**

We can estimate the sentiment of a word by averaging the sentiment scores of the reviews that the word appears in. For example, if the word "terribly" appears in two reviews with scores of 0 and one review with a score of 3, then the estimated sentiment score for the word "terribly" would be 1 (the average of 0, 0, and 3).

Once we have sentiment scores for a bunch of words, we can use those scores to predict the sentiment of a new movies review by averaging the sentiment scores of all the words in the movie review.

### 1. Word Sentiment Dictionary

Download the starter file `sentiment_analysis.py` and the three files with movie reviews described above from the course website.

Write a function `make_word_sentiment_dictionary`. The function should take the name of a file with movie reviews (in the format discussed above) as a parameter, and it should return a word sentiment dictionary. The keys in this dictionary should be individual words, and the values should be (small) dictionaries that assemble the following information about the word:

- the sum of the sentiment scores of the movie reviews in which the word appears (key: "total score")
- the number of movie reviews in which the word appears (key: "count")
- the average sentiment score of the movie reviews in which the word appears (key: "average score")

For example, here is what an excerpt of the word sentiment dictionary might look like:

```
{'nice': {'total score': 38, 'count': 14, 'average score': 2.7142857142857144},
'little': {'total score': 312, 'count': 154, 'average score': 2.0259740259740258},
'story': {'total score': 538, 'count': 251, 'average score': 2.143426294820717},
'process': {'total score': 41, 'count': 17, 'average score': 2.411764705882353},
...}
```

Note: convert all words to lower case before adding them to the dictionary. You can use the built-in string method `.lower()` for this purpose.

### 2. Predicting Sentiment Scores for Reviews

Now add a function `predict_sentiment_score`. Given a movie review as a string and a word sentiment dictionary, this function should return an estimated sentiment score for the review. It should estimate the score by averaging the scores of all the words in the review.

For example, given the review `"This movie is awesome !"` and the word sentiment dictionary created from the data in `movie_reviews_training.txt`, this function should return roughly `2.41`.

A movie review may contain some words that are not in the words sentiment dictionary that you've created. For those words, assume that they are neutral (i.e. that they have a score of 2).

### 3. Classifying Movie Reviews According to Their Sentiment

If you test a few of the reviews from the `movie_reviews_dev.txt` file, you will see that we are not able to totally accurately predict the sentiment scores. Most predictions your function makes are closer to the average than the actual scores of the reviews. However, if all we are interested in is whether or not a movie is rated positively, that may not matter. We may still be able to use the predicted scores to decide whether we should go see a movie or not.

Add a function `is_positive` that takes a movie review and a word sentiment dictionary, then classifies the review as either positive (i.e. humans would give it a score of 3 or 4) or not. The function should return a boolean value (i.e. `True` or `False`).

Try out your classification function on a few movie reviews from the `movie_reviews_dev.txt` file. The function won't get it right for all reviews, but it should get it right more often than not.

### 4. Fine Tune Your System

Now uncomment the definition of the function evaluate as well as the line in the testing area that calls it. This will run your sentiment analysis system on all 800 movie reviews from the file `movie_reviews_dev.txt` and print out some information on how many of them were classified correctly.

It should be fairly easy to classify more than 60% correctly. With further fine-tuning of `is_positive`, you may manage to get an accuracy as high as 75%.

### 5. Reflection

Reflect on your work on this project and write your thoughts in a file called `reflection.pdf`. Note this reflection should be quite substantial. Write in full sentences, in paragrpah/essay form. Include any references/citations you find as you are exploring your thoughts. Additionally, come to class ready to discuss your ideas. In particular, try to answer, based on your own opinion and in your own words:

- Can computers think?
- How do/can computers learn?
- What are some strengths and weaknesses of the method we used to simulate intelligent behavior in this project?
- How do you think the design of this program could be improved?

### 3. What to submit

What to submit You will need to submit the following files:

- `sentiment_analysis.py`
- `reflection.pdf`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:

a. A header comment, which includes your name and a brief description of the program.
b. Comments within the body of the code to further clarify how the program works.

2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code? Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?

a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting. Once you have checked these things, submit your files to *Project 4: Sentiment Analysis* on Gradescope.

**6. Grading guidelines**

- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.